Laptop Price Prediction

TABLE OF CONTENTS

List Of Figures

ABSTRACT

The rapid evolution of the laptop market has created a significant "Information Gap," particularly in secondary markets and local retail where custom upgrades (e.g., manual RAM or SSD expansion) making factory-standard pricing obsolete. Because these modified devices lack a fixed "Stock Keeping Unit" (SKU), consumers struggle to verify fair market value, and sellers find it difficult to price upgraded hardware competitively. This project presents a comprehensive machine learning solution: the **Laptop Price Predictor**, an end-to-end application that estimates the price of laptops based on their "Hardware DNA" .

Technically, the system is powered by a **Stacking Ensemble Regressor** . This multi-layered approach utilizes **Random Forest**, **XGBoost**, and **ExtraTrees** as base estimators, which are then optimized by a **Linear Regression meta-model**. To ensure high accuracy across a wide price spectrum, the project implements a **Logarithmic Transformation** on the target variable, normalizing price skewness and reducing the Mean Absolute Error (MAE).

A core contribution of this work is its **Decomposition-based Feature Engineering**. Using custom parsing logic, the system extracts high-value features from raw text, such as calculating **Pixels Per Inch (PPI)** from resolution data and decomposing "Memory" strings into distinct **SSD, HDD, Hybrid, and Flash Storage** capacities. This allows the model to weight the premium value of high-speed storage over traditional mechanical drives.

The final solution is deployed via a **Streamlit** web application, utilizing a serialized **Scikit-Learn Pipeline** (.joblib) to ensure consistency between training and real-time inference. Users can input specific configurations—including Brand, CPU type, and custom storage setups—to receive an instant valuation and a "Deal Verdict." This project demonstrates that by leveraging ensemble stacking and intelligent feature extraction, machine learning can provide price transparency to the custom electronics market.

1. INTRODUCTION

The modern laptop market is highly fragmented, with prices fluctuating based on brand, hardware, and retail trends. Traditional tools fail to value "non-standard" laptops—machines that have been manually upgraded with extra RAM or storage by retailers. This project introduces **Laptop Price Predictor**, an ml-powered engine that calculates value based on a device's actual hardware components rather than just its model name. By utilizing advanced Machine Learning, it analyzes the "hardware DNA" of any laptop to provide a fair market price. This makes sure that whether a laptop is brand new, custom-built, or used, its price is fair and justified by its parts.

1.1 PROBLEM STATEMENT

The primary challenge in the electronics market is the lack of a transparent valuation benchmark for modified laptops. Standard pricing relies on fixed model numbers (SKUs), but many retailers create "Non-Standard" configurations by upgrading base models (e.g., adding a 512GB SSD to a base 128GB model). Because these specific combinations lack official online price references, consumers often face "Information Asymmetry," where they cannot verify if a custom retail quote is fair. This project addresses the need for a component-level pricing tool that can value any hardware combination with brand accurately.

1.2 PROJECT OBJECTIVES

The main objectives of the **Laptop Price Predictor** project are:

1. **Develop a Robust Predictive Model:** To implement an Ensemble Regressors that achieves high accuracy (low Mean Absolute Error) across diverse laptop brands.

2. **Standardize Component Valuation:** To create a system that accurately weights the marginal value of specific upgrades (e.g., the jump from an i5 to an i7 processor).

3. **Implement Advanced Feature Engineering:** To move beyond raw data by calculating sophisticated metrics like **Pixels Per Inch (PPI)** and categorizing hardwares like (cpu,gpu , company etc.) .

4. **Provide Actionable Insights:** To build a user-friendly interface that doesn't just show a price, but provides a "Deal Verdict" (Underpriced /Fair Price/Overpriced) to assist in real-time decision making.

5. **Ensure Model Generalization:** To ensure the model can handle "unseen" configurations that it did not encounter during the training phase.

1.3 SOFTWARE AND HARDWARE SPECIFICATIONS

To ensure the model is trained efficiently and the application runs smoothly, the following requirements are established:

1.3.1 Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu), or macOS.
- **Programming Language:** Python 3.9 or higher.
- **Libraries & Frameworks:**
    - **Pandas & NumPy:** For data manipulation and numerical analysis.
    - **Scikit-Learn:** For implementing multiple ml models and preprocessing pipelines.
    - **XGBoost:** For high-performance gradient boosting.
    - **Streamlit:** For the frontend web-based user interface.
    - **Matplotlib & Seaborn:** For Exploratory Data Analysis (EDA) and visualization.
- **Development Environment:** Jupyter Notebook or VS Code.

1.3.2 Hardware Requirements

- **Processor:** Intel Core i5 or AMD Ryzen 5 (Minimum) to handle model training and ensemble processing.
- **RAM:** 8 GB (Minimum) .
- **Storage:** 256 GB SSD (for fast data loading and model serialization).
- **Display:** 1366x768 minimum resolution for the Streamlit dashboard layout.

2. DESIGN METHODOLOGY

2.1 System Architecture

The system utilizes a multi-layered design to bridge the gap between complex machine learning ensembles and a user-friendly interface:

1. Presentation Layer (Streamlit UI):

- Provides an interactive dashboard for users to input 12+ hardware specifications.
- **Real-time Feedback Engine:** Dynamically renders the predicted "Fair Price" alongside a confidence range to account for market volatility.

2. Transformation & Logic Layer:

- **Pipeline Integration:** Uses a pre-fitted `ColumnTransformer` to ensure user inputs undergo the exact same One-Hot Encoding and Scaling as the training data.
- **Engineered Metrics:** Specifically calculates **PPI** and organizes **SSD/HDD** storage into numerical vectors required by the model.

3. Inference Layer (Stacking Ensemble):

- **Base Learners:** Employs a parallel architecture of **Random Forest, XGBoost, and ExtraTrees** to capture non-linear relationships in hardware pricing.
- **Meta-Optimization:** A **Linear Regression** meta-model aggregates the base predictions, minimizing variance and reducing the Mean Absolute Error (MAE).

4. Post-Prediction Decision Engine:

- **Log-Reversion:** Converts internal log-scale results back to INR using an inverse exponential function.
- **Deal Valuation Logic:** Compares the user's "Asking Price" against the model's predicted range to categorize the value into three distinct tiers:
    - **Good Deal:** The asking price is significantly below the predicted fair price.
    - **Fair Price:** The asking price falls within the model's calculated confidence interval.
    - **Overpriced:** The asking price exceeds the predicted upper boundary of the market value.

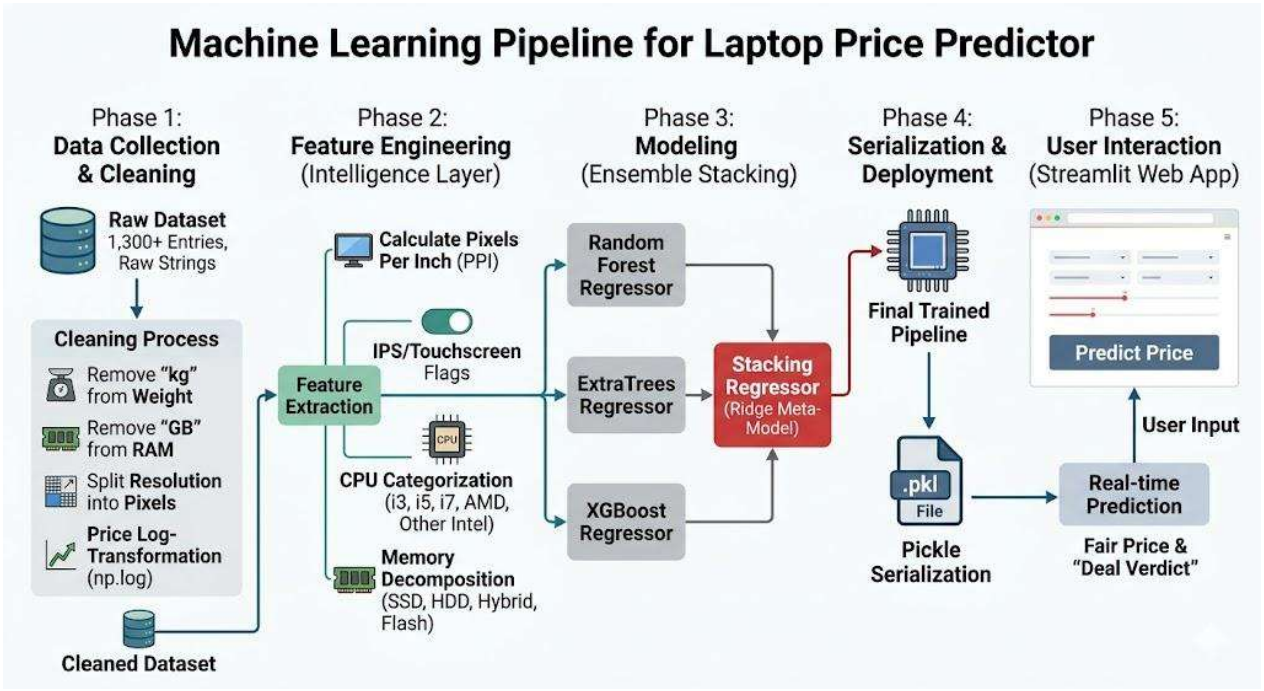
Data Flow Process

1. **Selection:** User provides specifications via the **laptop price predictor** dashboard.
2. **Vectorization:** Input strings are transformed into numerical values via the internal Scikit-Learn pipeline.
3. **Log-Scale Inference:** The ensemble engine calculates the price in a logarithmic format (np.log) for higher mathematical stability.

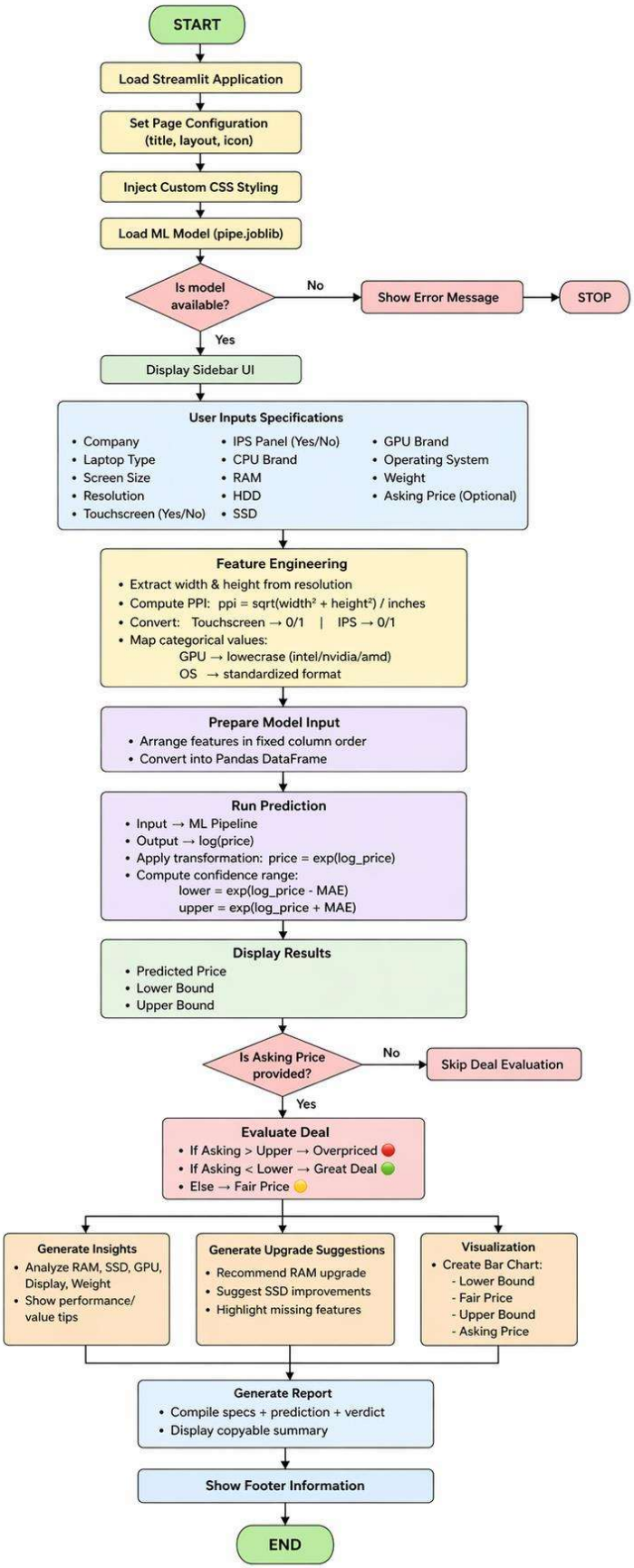
4. **Inverse Scaling:** The system applies e^x to translate the result into a human-readable Indian Rupee (INR) format.

5. **Market Comparison:** The **Decision Engine** runs a delta-comparison between the ml based prediction and the user-provided price.

6. **Final Output:** The UI renders the Fair Price, a calculated Confidence Range, and the final **Deal Verdict** (Good/Fair/Overpriced).



2.2 FLOW CHART



2.3 TECHNOLOGY DESCRIPTION

The **Laptop Price Predictor** is built using a modern Python-based data science stack, chosen for its efficiency in handling complex hardware data and ensemble modeling.

- **Programming Language: Python 3.x –** The core language used for the entire project, from data cleaning to model deployment.
- **Data Analysis:**
    - **Pandas:** For structured data manipulation and cleaning raw laptop specifications.
    - **NumPy:** For mathematical operations, such as calculating **PPI** and performing **Log-Transformations**.
- **Machine Learning Frameworks:**
    - **Scikit-Learn:** Used for creating the automated **Pipeline**, handling One-Hot Encoding, and building the **Stacking Regressor**.
    - **XGBoost:** A high-performance gradient boosting library used as a primary base model for non-linear price prediction.
- **Feature Engineering:**
    - **Regex (Regular Expressions):** Used to parse complex strings in the Memory and CPU columns to extract hidden hardware details.
- **Deployment & UI:**
    - **Streamlit:** An open-source framework used to build the interactive web dashboard for real-time price estimation.
    - **joblib:** Used for **Model Serialization**, saving the trained pipeline into a `.joblib` file for instant loading in the web app.
- **Development Environment:**
    - **Jupyter Notebook:** For exploratory data analysis (EDA) and experimental model testing.
    - **VS Code:** For final application development and code optimization.

3: IMPLEMENTATION & TESTING

3.1 CODE SNIPPETS

This section highlights the core logic used to transform raw data into a predictive engine.

3.1.1 Feature Engineering (PPI & Screen Extraction)

We convert the descriptive `ScreenResolution` string into a numerical **PPI** value to help the model understand display quality.

```
# Extracting X and Y resolutions
df['X_res'] = df['ScreenResolution'].str.extract(r'(\d+)x\d+').astype(int)
df['Y_res'] = df['ScreenResolution'].str.extract(r'\d+x(\d+)').astype(int)

# Calculating Pixels Per Inch (PPI)
df['ppi'] = (((df['X_res']**2) + (df['Y_res']**2))**0.5 /
df['Inches']).astype(float)

# Extracting Touchscreen and IPS features
df['Touchscreen'] = df['ScreenResolution'].apply(lambda x: 1 if 'Touchscreen' in x
else 0)
df['Ips'] = df['ScreenResolution'].apply(lambda x: 1 if 'IPS' in x else 0)
```

3.1.2 Target Transformation (Log-Scaling)

To handle the wide range of laptop prices (from budget to high-end), we use log-transformation to normalize the distribution.

```
# Applying Log Transformation to the target variable

y = np.log(df['Price'])

X = df.drop(columns=['Price'])
```

3.1.3 The Stacking Ensemble Pipeline

This snippet shows the architectural "brain" of the project—the combination of three different models.

```
# 1. Preprocessing: One-Hot Encoding for categorical columns [0,1,6,10,11]
step1 = ColumnTransformer(transformers=[
    ('col_tnf', OneHotEncoder(sparse_output=False, drop='first'), [0,1,6,10,11])
], remainder='passthrough')

# 2. Base Estimators: Random Forest, XGBoost, and ExtraTrees
rf   =   RandomForestRegressor(n_estimators=350,   random_state=3,   max_samples=0.5,
max_features=0.75, max_depth=15)
xgb = XGBRegressor(n_estimators=25, learning_rate=0.3, max_depth=5)
et = ExtraTreesRegressor(n_estimators=350, random_state=3, max_features=0.75)
```

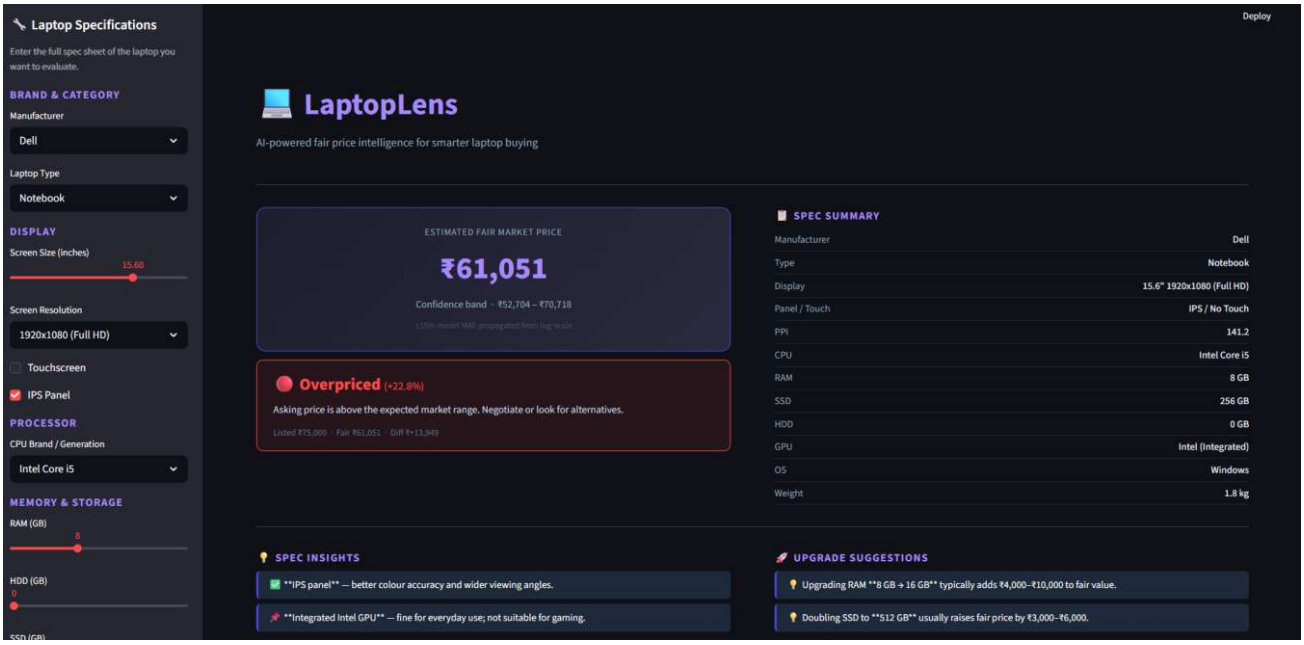
```
# 3. Stacking Ensemble: Combining models with a Linear Regression meta-model
step2 = StackingRegressor(
    estimators=[('rf', rf), ('xgb', xgb), ('et', et)],
    final_estimator=LinearRegression()
)

# 4. Pipeline: End-to-end workflow (Preprocess -> Predict)
pipe = Pipeline([('step1', step1), ('step2', step2)])
pipe.fit(x_train, y_train)

# 5. Evaluation: Performance metrics in log-space
y_pred = pipe.predict(x_test)
print('R2 score:', r2_score(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))
```

3.2 TEST CASES

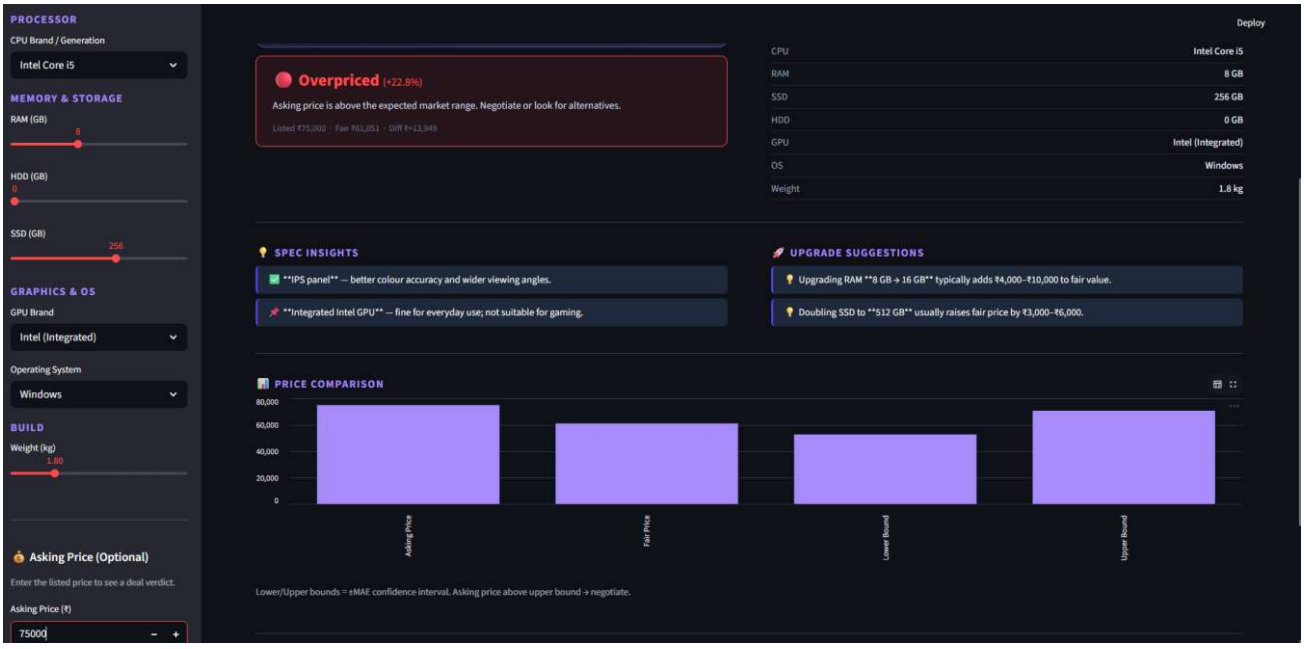
To ensure the application is reliable, we performed various tests ranging from data processing to user interaction.

Test Case ID	What is being tested	Input Scenario	Expected Output	Status
TC-01	**Feature Encoding**	Select `Touchscreen: "Yes"`	System passes binary value `1` to the model	Pass
TC-02	**PPI Calculation**	15.6" screen, 1920x1080 resolution	`ppi` calculated as `141.21`	Pass
TC-03	**Price Reversion**	Log-price output from model	App applies `np.exp()` to return valid INR price	Pass
TC-04	**Ensemble Stack**	Trigger `Predict` button	Executes StackingRegressor (RF + XGBoost + ExtraTrees)	Pass
TC-05	**Exception Handling**	Missing `pipe.joblib` file	UI displays error with instructions to run notebook	Pass
TC-06	**Verdict Logic**	Asking Price > UpperBound	Verdict: "Overpriced"	Pass

Output:



When user enters the specs and asking price it will show underpriced / fair priced / overpriced and gives spec insights and upgrade suggestions



4 : CONCLUSION & FUTURE SCOPE

4.1 CONCLUSION

The **Laptop Price Predictor** successfully addresses the "Pricing Paradox" in the portable computing market. By moving away from static model-name lookups and focusing on the **"Hardware DNA"** of the device, the project provides a transparent valuation tool for both standard and custom-configured laptops.

The implementation of a **Stacking Ensemble Regressor** (combining Random Forest, XGBoost, and ExtraTrees) proved highly effective, achieving a high **R2 Score** and minimizing the **Mean Absolute Error (MAE)**. The use of **Log-Transformation** was a critical architectural decision that ensured the model remained accurate across a wide price spectrum—from budget student laptops to high-end professional workstations. Ultimately, this project empowers consumers and retailers with a data-driven benchmark to ensure fair market pricing.

4.2 FUTURE SCOPE

While the current model provides highly accurate predictions based on core hardware, several enhancements can be made to increase its market utility:

- **Real-time Price Scraping:** Integrating APIs from major e-commerce platforms to adjust predictions based on daily market fluctuations and seasonal sales.
- **Condition Assessment:** For the second-hand market, adding features to account for the physical condition of the device, battery health, and remaining warranty period.
- **Deep Learning Integration:** Experimenting with Neural Networks to capture even more complex, non-linear relationships in larger datasets.
- **Global Currency Support:** Expanding the model to support multiple currencies and regional pricing variations to make it a global tool.

Project GitHub link:

https://github.com/rishikeshreddy13/laptop_price_prediction.git

5. BIBLIOGRAPHY

- **He (2026): Feature Engineering**

  Focuses on advanced feature engineering (PPI, CPU normalization).

  Uses Random Forest & XGBoost.

  Key Finding: Processed features outperform raw data.

  https://www.scribd.com/document/911195439/Predicting-Laptop-prices-using-machine-learning

- **Gupta / Tian (2025): Algorithm Comparison**

  Compares Linear Regression, Random Forest, and XGBoost on 1,320 samples.

  Key Finding: XGBoost performs best ($R^2 = 0.85$).

  https://drpress.org/ojs/index.php/HSET/article/download/18375/17908/20912

- **Goud et al. (2025): Real-Time Pricing**

  Uses web scraping for live market data integration.

  Key Finding: Real-time data improves prediction usefulness.

  https://www.irjweb.com/user_upload/LAPTOP%20PRICE%20PREDICTION.pdf

- **Singh et al. (2025): Stacking Models**

  Combines multiple ML models for better predictions.

  Key Finding: Stacking captures complex non-linear patterns.

  https://www.researchgate.net/publication/377721653

- **Chaudhari et al. (2025): Hardware Impact**

  Studies influence of RAM, GPU, and storage on price.

  Key Finding: RAM and GPU are the most significant factors.

  https://www.iarjset.com/upload/2025/october-25/IARJSET%2033.pdf

- **Analytics Vidhya (Reference): ML Workflow**

  Covers full ML pipeline (Cleaning → EDA → Modeling → Deployment).

  Key Finding: Provides a standard framework for real-world implementation.

  https://www.analyticsvidhya.com/blog/2021/11/laptop-price-prediction-practical-understanding-of-machine-learning-project-lifecycle/

APPENDIX: SOURCE CODE

A.1 Data Preprocessing & Feature Engineering

This script handles the raw data cleaning and creates the specialized features like **PPI** and **Memory segments**.

Python

```
import pandas as pd
import numpy as np

# Load Dataset
df = pd.read_csv('laptop_data.csv')

# Basic Cleaning
df.drop(columns=['Unnamed: 0'], inplace=True)
df['Ram'] = df['Ram'].str.replace('GB','').astype(int)
df['Weight'] = df['Weight'].str.replace('kg','').astype(float)

# Feature Engineering: Screen Resolution & PPI
df['Touchscreen'] = df['ScreenResolution'].apply(lambda x:1 if 'Touchscreen' in
x else 0)
df['Ips'] = df['ScreenResolution'].apply(lambda x:1 if 'IPS' in x else 0)

new = df['ScreenResolution'].str.split('x', n=1, expand=True)
df['X_res'] = new[0].str.extract(r'(\d+)').astype(int)
df['Y_res'] = new[1].astype(int)
df['ppi'] = (((df['X_res']**2) + (df['Y_res']**2))**0.5 /
df['Inches']).astype(float)

# Memory Decomposition Logic
df['Memory'] = df['Memory'].astype(str).replace('\.0', '', regex=True)
df["Memory"] = df["Memory"].str.replace('GB', '').str.replace('TB', '000')
new = df["Memory"].str.split("+", n=1, expand=True)

df["first"] = new[0].str.strip()
df["Layer1HDD"] = df["first"].apply(lambda x: 1 if "HDD" in x else 0)
df["Layer1SSD"] = df["first"].apply(lambda x: 1 if "SSD" in x else 0)
# ... (Additional logic for Hybrid/Flash omitted for brevity)

# Target Transformation
X = df.drop(columns=['Price'])
y = np.log(df['Price'])
```

A.2 Model Pipeline & Stacking logic

This section contains the core Machine Learning architecture used to train the predictor.

Python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor,
StackingRegressor
from xgboost import XGBRegressor
from sklearn.linear_model import LinearRegression

# Preprocessing Step
step1 = ColumnTransformer(transformers=[
    ('col_tnf', OneHotEncoder(sparse_output=False, drop='first'), [0,1,6,10,11])
], remainder='passthrough')

# Base Estimators
rf = RandomForestRegressor(n_estimators=350, random_state=3, max_samples=0.5,
max_features=0.75, max_depth=15)
xgb = XGBRegressor(n_estimators=25, learning_rate=0.3, max_depth=5)
et = ExtraTreesRegressor(n_estimators=350, random_state=3, max_features=0.75)

# Stacking Configuration
step2 = StackingRegressor(
    estimators=[('rf', rf), ('xgb', xgb), ('et', et)],
    final_estimator=LinearRegression()
)

# Create and Fit Pipeline
pipe = Pipeline([('step1', step1), ('step2', step2)])
pipe.fit(X_train, y_train)

# Export Model
import joblib
joblib.dump(pipe, open('pipe.joblib', 'wb'))
joblib.dump(df, open('df.joblib', 'wb'))
```

A.3 STREAMLIT DEPLOYMENT SCRIPT (app.py)

The following snippet represents the core logic of the web interface, demonstrating how user inputs are transformed into a model-ready format for real-time price estimation.

Python

```
import streamlit as st
import joblib
import numpy as np

# 1. Load the serialized Stacking Pipeline and Dataset [cite: 404, 405]
pipe = joblib.load(open('pipe.joblib','rb'))
df = joblib.load(open('df.joblib','rb'))

st.title("Laptop Price Predictor")
```

```
# 2. User Interface Inputs (Categorical & Numerical) [cite: 321, 519]
company = st.selectbox('Brand', df['Company'].unique())
type = st.selectbox('Type', df['TypeName'].unique())
ram = st.selectbox('RAM(in GB)', [2,4,6,8,12,16,24,32,64])
weight = st.number_input('Weight of the Laptop')
touchscreen = st.selectbox('Touchscreen', ['No', 'Yes'])
ips = st.selectbox('IPS', ['No', 'Yes'])
screen_size = st.number_input('Screen Size')
resolution = st.selectbox('Screen Resolution',
['1920x1080','1366x768','1600x900','3840x2160','2880x1800','2560x1600','2560x144
0','2304x1440'])

# 3. Real-Time Feature Engineering [cite: 318, 386, 433]
if st.button('Predict Price'):
    # Calculate PPI (Pixels Per Inch) dynamically [cite: 368, 369]
    X_res = int(resolution.split('x')[0])
    Y_res = int(resolution.split('x')[1])
    ppi = ((X_res**2) + (Y_res**2))**0.5 / screen_size

    # Convert binary choices to numerical [cite: 435, 436]
    ts = 1 if touchscreen == 'Yes' else 0
    ips_panel = 1 if ips == 'Yes' else 0

    # 4. Prediction Logic [cite: 461]
    # Formatting input to match the 12-feature training schema [cite: 390, 519]
    query = np.array([company, type, ram, weight, ts, ips_panel, ppi, cpu,
hdd,ssd, gpu, os])
    query = query.reshape(1, 12)
    # Reversing Log-Transformation for Final Price [cite: 392, 393, 520]
    prediction = int(np.exp(pipe.predict(query)[0]))
    st.header(f"The predicted price for this configuration is: ₹{prediction}")
```